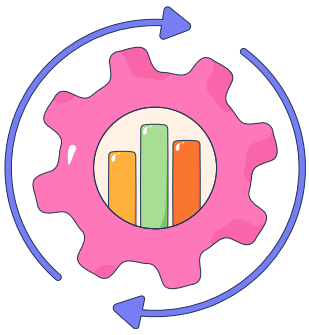
DynamoDB Throughput and Modes

Explore how DynamoDB manages throughput with provisioned and on-demand modes to optimize costs and performance. Understand auto scaling, warm throughput, burst capacity, and adaptive capacity features. Learn to design high-cardinality partition keys and handle hot partitions to maintain scalability and reduce throttling in your AWS applications.

Unlike traditional databases, DynamoDB offers two distinct capacity modes—provisioned and on-demand—to fit different workload profiles. Each mode had its own implications for performance, predictability, and cost.

AWS also provides several built-in mechanisms to help applications respond to fluctuating traffic without manual intervention.



Provisioned capacity mode

In provisioned capacity mode, we explicitly define how much read and write throughput our table needs. This involves setting:

Ask

- **Read Capacity Units (RCUs):** Each RCU allows one strongly consistent read per second for an item up to 4 KB in size, or two eventually consistent

reads.

- **Write Capacity Units (WCUs):** Each WCU allows one standard write per second for an item up to 1 KB.

Requests are throttled if our workload exceeds the provisioned limits and we don't have burst capacity available. Provisioned mode worked best for applications with predictable traffic patterns or those requiring tight cost control through pre-set limits.

Provisioned capacity mode

Throughput management with auto scaling

For tables using provisioned capacity, AWS offers *auto scaling* to dynamically adjust RCUs and WCUs in response to traffic patterns. Auto scaling works by:

- Monitoring CloudWatch metrics like `ConsumedReadCapacityUnits` and `ConsumedWriteCapacityUnits`.
- Adjusting throughput based on a target utilization percentage (typicall between 70–90%).

Ask

- Automatically increasing capacity during traffic spikes and decreasing it when demand falls, all while staying within your defined min/max limits.

Throughput management of DynamoDB table using AutoScaling

This helps keep the application responsive without requiring constant manual tuning, while also ensuring we are not overpaying for unused capacity during quiet periods.

On-demand capacity mode

On the other hand, on-demand capacity mode behaves like a truly serverless solution. We don't need to predict traffic or manually set RCUs or WCUs. DynamoDB automatically scales the table's capacity up and down based on the number of incoming reads and writes.

Ask

In this mode, we only pay for what you use, with no risk of manual under- or over-provisioning. It's ideal for spiky or unpredictable workloads, such as social media

apps, gaming backends, or retail sites during flash sales.

While the per-request cost may be slightly higher compared to provisioned mode, the reduction in operational overhead and the near-elimination of throttling risk can make it a compelling choice. Still, if the application traffic grows too quickly beyond internal scaling limits, throttling can occur even in on-demand mode, but it's much rarer.

On-demand capacity mode

Note: When the application exceeds the maximum throughput, DynamoDB throttles the request and returns a `ThrottlingException` exception. We can use this exception to take appropriate action.

Performance enhancements

Beyond basic capacity settings, DynamoDB includes several internal optimizations that help maintain fast and reliable performance even under workloads.

Ask

Warm throughput

Warm throughput refers to DynamoDB's strategy of keeping partitions "hot" when they are regularly accessed.

- A warm partition has memory and routing resources already active.
- Cold partitions (rarely accessed ones) may experience slightly higher latency initially because they need to "warm up".

DynamoDB uses warm throughout to deliver *consistent low-latency responses* for applications with steady access patterns.

Warm throughput

Burst capacity

Burst capacity allows DynamoDB tables to temporarily consume throughput beyond the provisioned limit if unused capacity has been accrued over the past minutes. This lets applications survive short-term surges in traffic—like a s batch of user logins or a viral post—without throttling immediately. It's like a "credit" system for throughput.

Ask

Burst capacity

Burst capacity is particularly valuable for sudden, unpredictable spikes.

Adaptive capacity

DynamoDB's adaptive capacity automatically redistributes partition throughput internally. Imagine one user becomes incredibly active, generating most of the read/write load. Rather than throttling that hot partition, DynamoDB can borrow unused capacity from less busy partitions. This happens automatically and does not require manual re-sharding or manual intervention.

Adaptive capacity helps:

- Reduce "hot partition" problems.
- Maintain consistent performance across your workload.

Ask

Adaptive provisioned capacity in DynamoDB

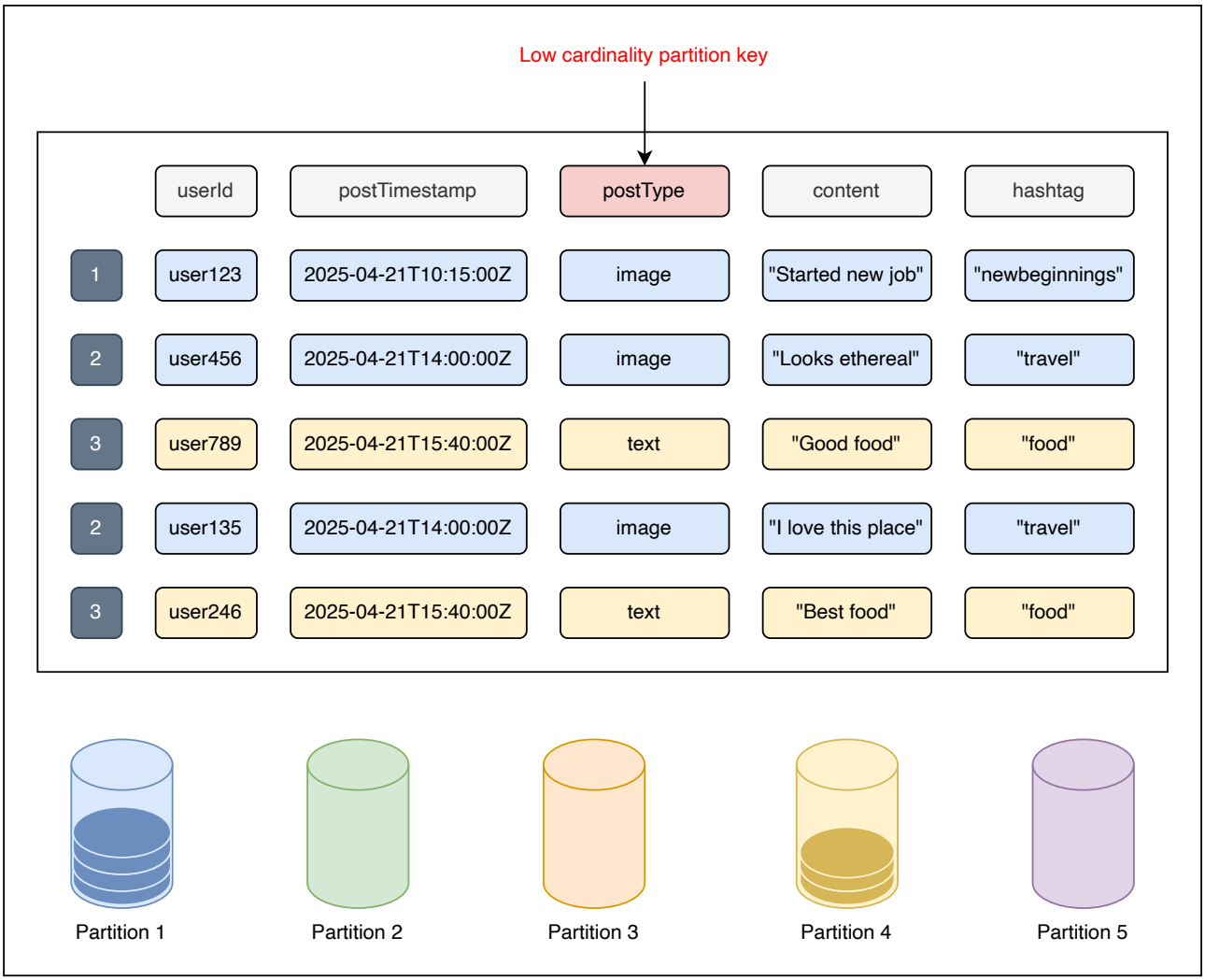
Designing partition keys: Low vs. high cardinality

Choosing the right partition key is essential to ensuring balanced throughput distribution in DynamoDB. Cardinality refers to the number of distinct values a key can take.

- Low-cardinality partition keys have very few unique values. For example, using our social media example, let's assume we use `"postType"` as a partition key. This seems logical at first—grouping posts by type—but in practice, it would cause serious performance issues, as there are only a handful of distinct post types (maybe 5–10 total). Millions of posts would all pile into a few partitions, creating hot partitions and throttling issues.
- High-cardinality partition keys have many unique values. For example, using `"UserId"` or `"OrderId"` as partition keys ensure that traffic is naturally distributed across several partitions, leading to better performance and reduced risk of throttling.

In general, high-cardinality keys are preferred for write-intensive and read-intensive workloads, as they enable DynamoDB to spread requests evenly across many partitions.

Ask



1 / 2

Low cardinality partition key

> **Tip:** When designing a table:
>
> - Favor keys that are diverse and unpredictable.
> - Avoid using timestamp-only, sequential numbers, or small categorical values unless additional randomization or sharding strategies are applied.

Ask

Hot partitions in DynamoDB

In DynamoDB, a **hot partition** happens when too much read or write traffic is concentrated on a small number of partitions or even a single partition instead of being evenly distributed across all partitions in the table.

Because each partition has its own throughput limits, overwhelming one partition while others stay idle can cause throttling, higher latencies, and application performance issues, even if the table overall has enough capacity.

Why hot partitions happen?

DynamoDB partitions the data internally based on the partition key. If the application design causes many read or write operations to target the same partition key (or a small group of keys), we unintentionally focus traffic on a few partitions. Common causes include:

- Using low-cardinality partition keys (e.g., storing all users under a single region code).
- Using sequential keys (e.g., timestamps, counters) that cause newly written data to land on the same partition.
- Spiky access patterns like when many users try to access the same product page during a flash sale.

For example, a sudden spike in the request for posts by `user123`.

Ask

Hot partition in DynamoDB

How to handle hot partitions

Even with DynamoDB's internal optimizations like adaptive capacity, extreme workloads can still create "hot partitions"—situations where a single partition receives a disproportionate share of the read/write load.

Applications need to proactively design for these scenarios to avoid throttling and latency issues. Key techniques to handle hot partitions include:

- **Add random suffixes:** Append random characters or sharded suffixes to frequently accessed keys (e.g., `user123-a`, `user123-b`, etc.) to distribute access across multiple partitions. Application logic then aggregates the results.
- **Use DAX (DynamoDB Accelerator):** Integrating DAX helps absorb high-read traffic bursts by caching popular items in memory, drastically reducing the pressure on the backend table.
- **Implement backoff and retry strategies:** When a request is throttled, use exponential backoff with jitter to reduce collision probability. This ensures

Ask